



Algorithm

2024 Spring

Assignment #4: Graph Algorithms

Yunyong Ko

School of Computer Science and Engineering

Chung-Ang University

Assignment #4: Graph Algorithms

□ Overview: implementing two graph algorithms

- (1) Minimum spanning tree: **Prim's algorithm**
 - Binary heap-based priority queue
- (2) All-pairs shortest paths: **Floyd-Warshall algorithm**
 - Adjacency-matrix representation

Algorithm to implement (1/2)

□ Prim's Algorithm

■ Binary heap-based priority queue

- Extract-Min
- Decrease-Key
- Insert, Min-Heapify

```
MST-PRIM( $G, w, r$ )
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$  Insert
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$  Extract-Min
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$  Decrease-Key
```

Functions to implement

❑ Extract-Min (deletion): $O(\log n)$

- (1) Extract the element with the highest priority (minimum)
- (2) Reorganize the priority queue using **Min-Heapify()**

❑ Min-Heapify: $O(\log n)$

- Revise the Max-Heapify function

❑ Should be revised

```
HEAP-EXTRACT-MAX(A)
1  if  $A.heap-size < 1$ 
2    error "heap underflow"
3   $max = A[1]$ 
4   $A[1] = A[A.heap-size]$ 
5   $A.heap-size = A.heap-size - 1$ 
6  MAX-HEAPIFY(A, 1)
7  return  $max$ 
```

```
MAX-HEAPIFY(A, i)
1   $l = LEFT(i)$ 
2   $r = RIGHT(i)$ 
3  if  $l \leq A.heap-size$  and  $A[l] > A[i]$ 
4     $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.heap-size$  and  $A[r] > A[largest]$ 
7     $largest = r$ 
8  if  $largest \neq i$ 
9    exchange  $A[i]$  with  $A[largest]$ 
10   MAX-HEAPIFY(A,  $largest$ )
```

Please refer to the lecture 5-2 (heap & priority queue)

Functions to implement

❑ Decrease-Key: $O(\log n)$

- Decrease the value of x 's key to the new value k
- Traverse a path from x to the root to find a proper place for the newly decreased key
 - ❑ (1) Compare its key with its parent's key
 - ❑ (2) Decide whether exchanging their keys

❑ Insertion: $O(\log n)$

- (1) Expand the min-heap by adding a new leaf node
- (2) Call $\text{Decrease_Key}(A, A.\text{size}, \text{key})$
 - ❑ To set the value of the new leaf node's key
 - ❑ To maintain the min-heap property

HEAP-INCREASE-KEY(A, i, key)

```
1 if key < A[i]
2   error "new key is smaller than current key"
3 A[i] = key
4 while i > 1 and A[PARENT(i)] < A[i]
5   exchange A[i] with A[PARENT(i)]
6   i = PARENT(i)
```

Revise the Increase-Key function

MAX-HEAP-INSERT(A, key)

```
1 A.heap-size = A.heap-size + 1
2 A[A.heap-size] = -∞
3 HEAP-INCREASE-KEY(A, A.heap-size, key)
```

❑ Should be revised

Please refer to the lecture 5-2 (heap & priority queue)

Algorithm to implement (2/2)

□ Floyd-Warshall Algorithm

- Adjacency-matrix representation

FLOYD-WARSHALL(W)

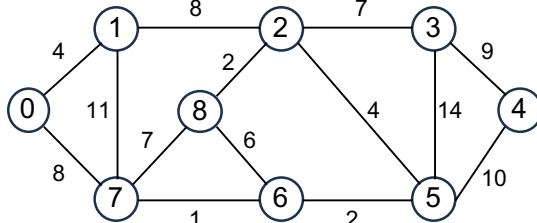
```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```

Description: Input and Output Files (1/2)

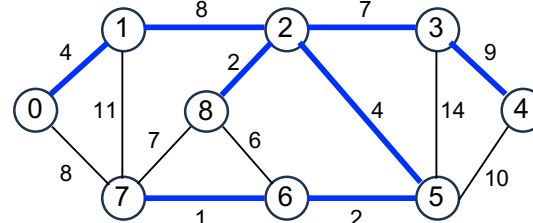
- ❑ Input file (**undirected** graph)
- ❑ Output file (Prim's algorithm)

```
[num_vertices]<tab>[num_edges]<tab>[start vertex ID]  
[vertex ID] <tab> [vertex ID] <tab> [weight]  
...  
[vertex ID] <tab> [vertex ID] <tab> [weight]
```

```
[vertex ID]<tab>[predecessor]  
...  
[vertex ID]<tab>[predecessor]
```



9	14	0
0	1	4
0	7	8
1	2	8
1	7	11
2	3	7
2	5	4
2	8	2
3	4	9
3	5	14
4	5	10
5	6	2
6	7	1
6	8	6
7	8	7

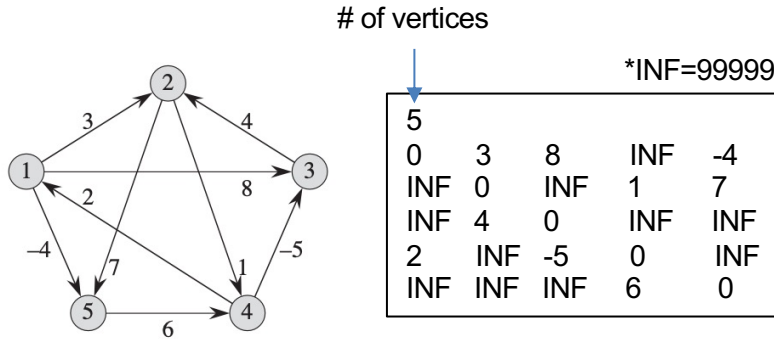


0	NIL
1	0
2	1
3	2
4	3
5	2
6	5
7	6
8	2

Description: Input and Output Files (2/2)

□ Input file (**directed** graph)

□ Output file (Floyd-Warshall)



$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{weight of } (i, j) & \text{if } i \neq j, (i, j) \in E, \\ \infty & \text{if } i \neq j, (i, j) \notin E. \end{cases}$$



D: shortest-path weight matrix

P: predecessor matrix

D	5	# of vertices				
0	1	-3	2	-4		
3	0	-4	1	-1		
7	4	0	5	3		*NIL=-1
2	-1	-5	0	-2		
8	5	1	6	0		
P	5	# of vertices				
NIL	3	4	5	1		
4	NIL	4	2	1		
4	3	NIL	2	1		
4	3	4	NIL	1		
4	3	4	5	NIL		

$D[i][j] = \delta(i, j)$: the weight of a shortest-path from vertex i to vertex j

$P[i][j]$ = the predecessor of vertex j on a shortest-path from vertex i to vertex j

□ Writing a makefile

```
all: compile

compile: graph.c
    gcc graph.c -o graph

run: graph
    ./graph input_sp.txt input_mst.txt output_sp.txt output_mst.txt

clean: graph
    rm graph output_sp.txt output_mst.txt
```

Makefile

'make' will compile your code and generate the executable file
(e.g., gcc graph.c -o graph)

'make run' will run your program
(./ graph input_sp.txt input_mst.txt output_sp.txt output_mst.txt)

4 filenames: 2 input files and 2 output files

'make clean' will delete the executable file and output file
(e.g., rm graph output_sp.txt output_mst.txt)

**Please make sure that your code is compiled and run successfully,
and the results in the output file are correct**

Evaluation

- ☐ Results on test input files
- ☐ Comments for all function that you implemented
- ☐ Time complexity of the functions that you implemented
- ☐ Makefile

Submission Guideline

□ Submission

- What: **Compressed file (Source code, input file, makefile)**
 - **Filename: Assignment4_StudentID**
 - ***Compress the three files together**, **NOT** your workspace directory
 - Or name your workspace directory as **Assignment4_studentID**
- Where: **Assignment board on e-class**
- Deadline: **11:59 PM, June 14th (Friday), 2024**
 - Late submission: ~ June 16th (Sunday), 2024 (+1 day: 75%, +2 day: 50%)

□ Q&A

- Office hour: 3:00 PM, Tuesday
- Contact Information
 - Email: yyko@cau.ac.kr
 - Tel: 02-820-5507

Q&A:
Assignment #4: Graph Algorithms